Languages and Machines

L0. Introduction

Jorge A. Pérez

Bernoulli Institute for Mathematics, Computer Science, and AI
University of Groningen, Groningen, the Netherlands

April 13, 2026

L&M, In a Nutshell

- A course on **automata theory and formal languages**
- Lecturers: Dr. Dan Frumin and Dr. Jorge Pérez

L&M, In a Nutshell

- A course on **automata theory and formal languages**
- Lecturers: Dr. Dan Frumin and Dr. Jorge Pérez
- Lectures (at least one per week) and tutorials (one per week)
- We assume you have passed (and still remember!)
  - Introduction to Logic
  - Discrete Structures (in particular: the induction principle)

L&M, In a Nutshell

- A course on **automata theory and formal languages**
- Lecturers: Dr. Dan Frumin and Dr. Jorge Pérez
- Lectures (at least one per week) and tutorials (one per week)
- We assume you have passed (and still remember!)
  - Introduction to Logic
  - Discrete Structures (in particular: the induction principle)
- Assessment (revised this year):
  - A final exam (3h): you pass the course if you pass the exam
  - Two individual homeworks: preparation for the exam, can improve your final grade

L&M, In a Nutshell

- A course on **automata theory and formal languages**
- Lecturers: Dr. Dan Frumin and Dr. Jorge Pérez
- Lectures (at least one per week) and tutorials (one per week)
- We assume you have passed (and still remember!)
  - Introduction to Logic
  - Discrete Structures (in particular: the induction principle)
- Assessment (revised this year):
  - A final exam (3h): you pass the course if you pass the exam
  - Two individual homeworks: preparation for the exam, can improve your final grade
- Self-study is important!
- Helpdesk email: `lm26.rug.cs@gmail.com`

The Foundations of Computation

Basic questions:

- What does it mean for a program to be computable?

The Foundations of Computation

Basic questions:

- What does it mean for a program to be computable?
- Are there non computable programs / functions?
- Computational power $\leftrightarrow$ Programming constructs?

The Foundations of Computation

Basic questions:

- What does it mean for a program to be computable?
- Are there non computable programs / functions?
- Computational power $\leftrightarrow$ Programming constructs?

Looking for answers $\rightarrow$ **Fundamental concepts**

- State
- Transition
- Non-determinism
- Undecidability
- $\cdots$

Persistent concepts, despite many (and frequent) technology changes

Models of Computation

We have different **machines**, in order of increasing power:

(a) Finite Memory:
Finite automata, regular expressions

(b) Finite Memory with stack:
Pushdown automata

(c) Unrestricted:
Turing machines (terminating and non-terminating)

Grammars and Languages

Our interest is in **languages**, sets of strings.
Languages are neatly characterized by **grammars**.

The **Chomsky hierarchy** organizes grammars in order of increasing complexity:

(i) Regular grammars
(ii) Context-free grammars
(iii) Unrestricted grammars

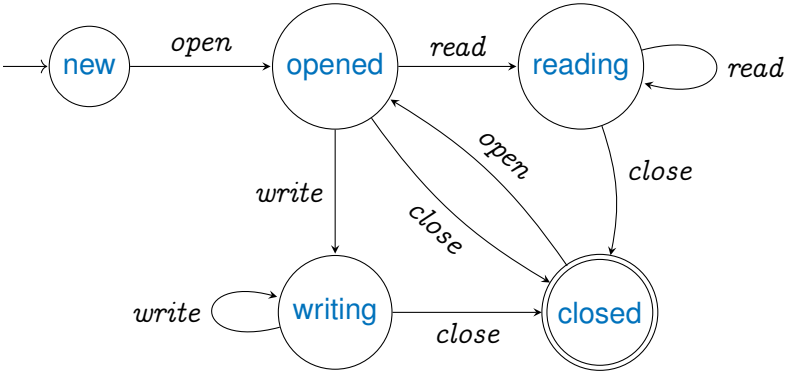
Grammars and Machines

Alternative ways of approaching languages

- Superficially very different
- Parsing a sentence in a language is quite similar to computation in a machine
- Grammar types (i)-(iii) are **equivalent** to machines (a)-(c)!

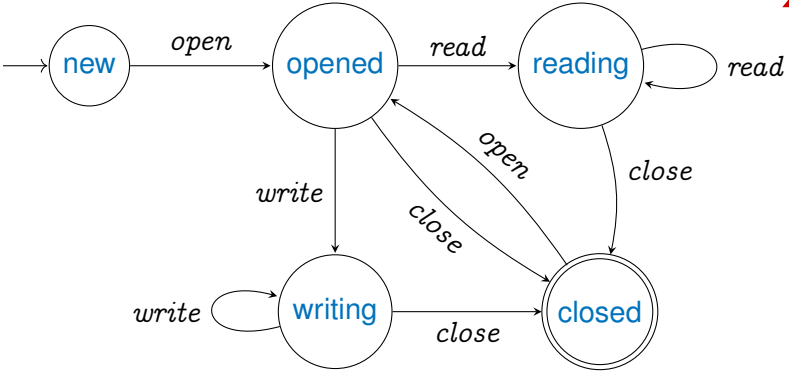
This is a bit abstract — let us examine an example.

A file and its operations



We have states and transitions between them.

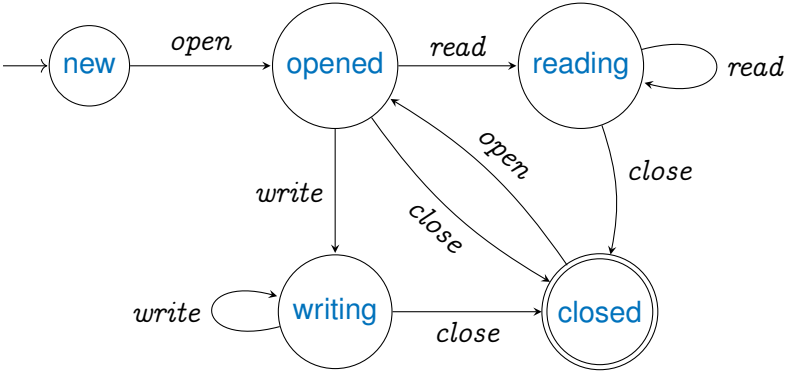
A file and its operations



We have states and transitions between them. Some properties:

- We must create a file before we can use it
- The file must be opened before reading or writing
- If we are reading the file, we must close it before writing on it (and vice versa)
- It's OK to open the file and to close it immediately after

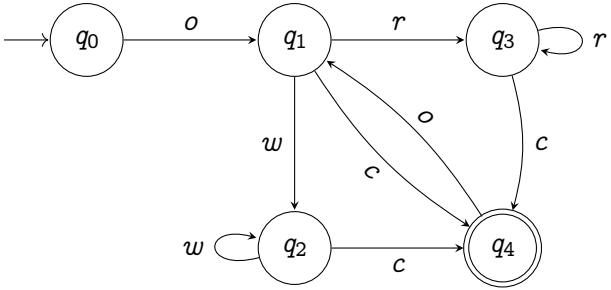
A file and its operations



We have states and transitions between them. Some sequences of operations:

- Valid: $open \cdot read \cdot read \cdot close$, $open \cdot read \cdot close \cdot open \cdot write \cdot close$
- Invalid: $open \cdot close \cdot read$, $read \cdot read \cdot close$, $open \cdot read \cdot close \cdot open$

State-based systems



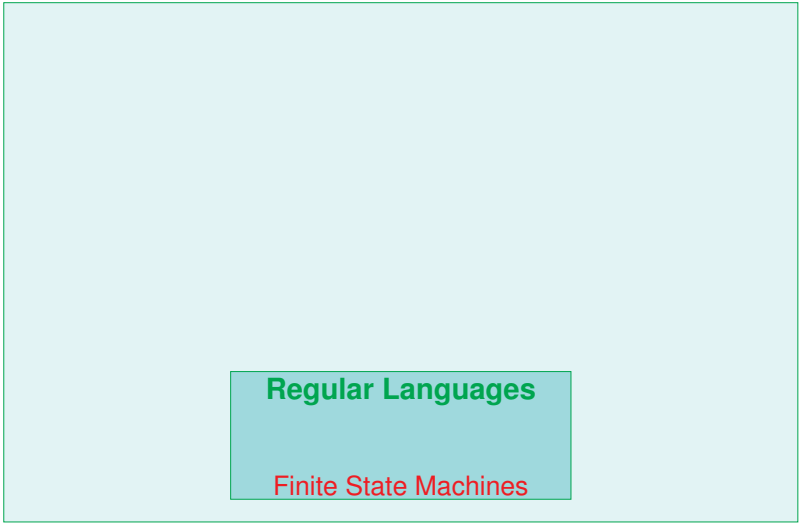
- Above, a representation of a machine with states q_0, q_1, q_2, q_3, and q_4. Transitions between states allow us to recognize symbols o, r, w, c. Two distinguished states: q_0 (the initial state) and q_4 (the final state).
- The machine recognizes a certain language: a set of strings.
  - The language includes strings like 'oc', '$orrrc$' and '$orcowwc$'
  - Strings such as '$rrww$' nor '$orwc$' are not part of the language

Many applications

State-based specifications are everywhere in CS:

- Programming language design and implementation
  (Compiler construction, domain specific languages, etc)
- Software and hardware verification
  (Model checking, run-time verification, etc)
- Learning and AI
- Bioinformatics
- Security
- ...

The Course, In a Nutshell



The Course, In a Nutshell

Regular Languages

Regular Grammars
Regular Expressions
Finite State Machines

Generator

Recognizer

The Course, In a Nutshell

Context-Free Languages
Context-Free Grammars
Pushdown Machines

Regular Languages
Regular Grammars
Regular Expressions
Finite State Machines

Generator

Recognizer

The Course, In a Nutshell

Context-Sensitive Languages
Context-Sensitive Grammars
Linearly Bounded Machines

Context-Free Languages
Context-Free Grammars
Pushdown Machines

Regular Languages
Regular Grammars
Regular Expressions
Finite State Machines

Generator

Recognizer

The Course, In a Nutshell

Decidable Languages
Terminating Turing Machines

Context-Sensitive Languages
Context-Sensitive Grammars
Linearly Bounded Machines

Context-Free Languages
Context-Free Grammars
Pushdown Machines

Regular Languages
Regular Grammars
Regular Expressions
Finite State Machines

Generator

Recognizer

The Course, In a Nutshell

Semi Decidable Languages
Unrestricted Grammars
Turing Machines

Decidable Languages
Terminating Turing Machines

Context-Sensitive Languages
Context-Sensitive Grammars
Linearly Bounded Machines

Context-Free Languages
Context-Free Grammars
Pushdown Machines

Regular Languages
Regular Grammars
Regular Expressions
Finite State Machines

Generator

Recognizer

The Course, In a Nutshell

Undecidable Languages

Semi Decidable Languages
Unrestricted Grammars
Turing Machines

Decidable Languages
Terminating Turing Machines

Context-Sensitive Languages
Context-Sensitive Grammars
Linearly Bounded Machines

Context-Free Languages
Context-Free Grammars
Pushdown Machines

Regular Languages
Regular Grammars
Regular Expressions
Finite State Machines

Generator

Recognizer

The Course, In a Nutshell

Expressiveness **Simplicity / Efficiency**

Regular

Context-free

Context-sensitive

Decidable

Semi-decidable

The Course, In a Nutshell

Expressiveness **Simplicity / Efficiency**

Regular

Context-free

Context-sensitive

Decidable

Semi-decidable

The Course, In a Nutshell

	Expressiveness	Simplicity / Efficiency
Regular	↓	↑
Context-free		
Context-sensitive		
Decidable		
Semi-decidable		

Overview

In this course, we will describe, analyse, and classify the languages that can be read by machines and the machines that can read them.

The course does not concern the interpretation of such languages.

Roughly, seven parts:

1. Regular and context-free languages
2. Finite state machines
3. Properties of regular languages
4. Pushdown machines
5. Properties of context-free languages
6. Turing machines and decidability issues
7. Non-context-free languages

Learning Outcomes (Ocasys)

1. The student learns to understand and apply:
   (a) The basic theory of finite state, pushdown, and Turing machines, and of the regular, context-free, and decidable and semi-decidable languages.
   (b) The relationships between machines and languages, and the translation algorithms between the various representations (e.g. regular expressions, normal forms of grammars).
2. The student obtains an elementary understanding of decidability, undecidability, semi-decidability, computability, time complexity, the classes P and NP, and the Chomsky hierarchy.

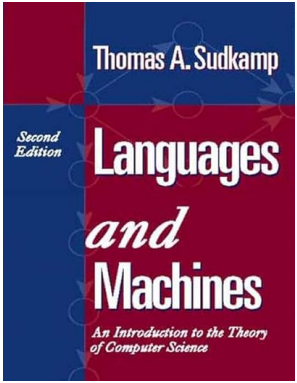
Material

"If it ain't broke, don't fix it"

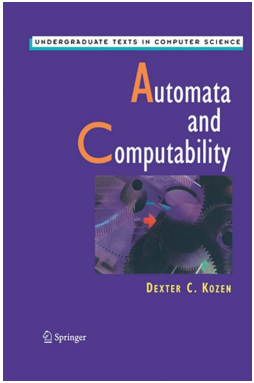
- Lecture Notes (reader) "Languages and Machines" by Wim Hesselink.
  Many exercises, some of which are discussed at the tutorials.
  PDF available in Brightspace: you comments are welcome!
- The reader is continuously revised: make sure to get the latest version!

The reader is our main reference, but it is not a textbook. Many good textbooks around!

Two Recommended Textbooks



Languages and Machines: An Introduction to the Theory of Computer Science
by Thomas A. Sudkamp



Automata and Computability
by Dexter C. Kozen

Teaching Method

On our side:

- In-person lectures (usually twice per week).
- Tutorials (once per week).
  Teaching Assistants:
  Helena Jager, Alexandra Vasile, and Ileana Untaru

Schedules in Brightspace (subject to changes).

Teaching Method

On our side:

- In-person lectures (usually twice per week).
- Tutorials (once per week).
  Teaching Assistants:
  Helena Jager, Alexandra Vasile, and Ileana Untaru

Schedules in Brightspace (subject to changes).

On **your side**: self-study!

Self-Study

Before **each lecture**:

- Study the reader and consult textbooks as needed
- Identify potential questions

Before **each tutorial**:

- Work on the suggested exercises — it is good to “get stuck”!
- Contact TAs about potential questions or specific exercises to discuss

Self-Study

Before **each lecture**:

- Study the reader and consult textbooks as needed
- Identify potential questions

Before **each tutorial**:

- Work on the suggested exercises — it is good to “get stuck”!
- Contact TAs about potential questions or specific exercises to discuss

Self-study also means:

- Attending lectures and tutorials
- Handing homeworks on time
- Look into topics not covered (or partially covered) in the course
- Providing constructive feedback to the lecturer and TAs

Grading (Ocasys)

Components

1. E: Exam
2. H_1, H_2: Two individual homeworks
3. $HWE = 0.7 \cdot E + 0.14 \cdot H_1 + 0.16 \cdot H_2$

Your Final Grade

$$F = \begin{cases} E & \text{if } E < 5.75 \\ \max(E, HWE) & \text{otherwise} \end{cases}$$

Hence, you pass the course if $E \geq 5.75$.
If you pass, homeworks can help you improve the final grade.

Note: H_1, H_2 do not count at the resit, nor can they be transferred to the future.

Homeworks and Exam

Two different (and complementary) ways of assessing your learning process

- Homeworks are intended to cover selected portions of the content of the course
  In contrast, the exam is meant to cover the entire course content
- By design, homeworks allow you to reflect about the topics and your understanding of them, and to discuss with TAs and fellow students (more on this later)

Homeworks and Exam

Two different (and complementary) ways of assessing your learning process

- Homeworks are intended to cover selected portions of the content of the course
  In contrast, the exam is meant to cover the entire course content
- By design, homeworks allow you to reflect about the topics and your understanding of them, and to discuss with TAs and fellow students (more on this later)
- In contrast, the exam comes with specific requirements in time and space

Homeworks and Exam

Two different (and complementary) ways of assessing your learning process

- Homeworks are intended to cover selected portions of the content of the course
  In contrast, the exam is meant to cover the entire course content
- By design, homeworks allow you to reflect about the topics and your understanding of them, and to discuss with TAs and fellow students (more on this later)
- In contrast, the exam comes with specific requirements in time and space
- Accordingly, exercises in homeworks and exam are necessarily different in nature, goal, and scope
- Exam questions are designed to assess essential and representative concepts, while enabling fair, efficient grading

Homeworks and Exam

Two different (and complementary) ways of assessing your learning process

- Homeworks are intended to cover selected portions of the content of the course
  In contrast, the exam is meant to cover the entire course content
- By design, homeworks allow you to reflect about the topics and your understanding of them, and to discuss with TAs and fellow students (more on this later)
- In contrast, the exam comes with specific requirements in time and space
- Accordingly, exercises in homeworks and exam are necessarily different in nature, goal, and scope
- Exam questions are designed to assess essential and representative concepts, while enabling fair, efficient grading
- Although the exam may include topics/questions you have encountered in the homeworks, this is not necessarily the case!

Important Dates

Two individual homeworks:

- Deadline on Fridays, 10h
- See Brightspace for dates, instructions, and updates.

Exam and Resit

- See the rooster for dates / times.

Contact Us

1. General questions: send us an email

   lm26.rug.cs@gmail.com

2. Specific questions, feedback, requests for (online) meetings
   j.a.perez[at]rug.nl, d.frumin[at]rug.nl

You can always reach out to the academic advisor:

- Email: academicadvisor.cs[at]rug.nl

Plagiarism and Scientific Integrity

Study Guide Computer Science, section on “Fraud Prevention & Scientific Integrity” — https://student.portal.rug.nl/infonet/studenten/fse/programmes/bsc-cs/:

> *Plagiarism is not accepted at this university nor elsewhere in the scientific community.*
> *In all cases in which plagiarism is found or suspected, the examiner will inform the Board of Examiners.*
>
> *Possible consequences:*
>
> - *Warning*
> - *Exclusion from exams for the relevant course for 1 academic year*
> - *Exclusion from exams for several courses for 1 academic year*
> - *Exclusion from programme*

Plagiarism and Scientific Integrity

- While discussions among students are allowed, plagiarism is not.

Plagiarism and Scientific Integrity

- While discussions among students are allowed, plagiarism is not.
- Homeworks are strictly individual. This doesn't exclude the possibility of discussing homework exercises with your fellow students.
- Discussions are encouraged: they can be useful to clarify concepts and rule out misconceptions. No need to refer to specific exercises.

Plagiarism and Scientific Integrity

- While discussions among students are allowed, plagiarism is not.
- Homeworks are strictly individual. This doesn't exclude the possibility of discussing homework exercises with your fellow students.
- Discussions are encouraged: they can be useful to clarify concepts and rule out misconceptions. No need to refer to specific exercises.
- When in doubt, contact the lecturers in advance (**before** handing in your solutions). Declaring collaborations (and their nature) is also possible.
- Once again, plagiarism suspicions detected **after** receiving homework solutions will always be forwarded to the BoE.
- Last but not least: The use of AI tools (such as ChatGPT) for attempting homework exercises is discouraged and disallowed.

The End